

IBVAP

Research & References

Smart India Hackathon 2026

Problem Statement 26187

Ministry of Home Affairs / Sashastra Seema Bal

This document provides all research references, academic papers, documentation links, and resources used in the design and development of the IBVAP platform.

1. Object Detection & Tracking Research

1.1 YOLO11 (Ultralytics)

[1] YOLO11: Real-Time Object Detection

Source: *Ultralytics Official Documentation*

URL: <https://docs.ultralytics.com/models/yolo11/>

State-of-the-art real-time object detection model. Used as the primary detector for 8 COCO classes (person, bicycle, car, motorcycle, bus, train, truck, boat).

[2] Ultralytics GitHub Repository

Source: *GitHub*

URL: <https://github.com/ultralytics/ultralytics>

Open-source implementation of YOLO11 with CUDA/CPU auto-fallback, NMS optimization, and export capabilities.

[3] YOLOv11 Performance Benchmarks

Source: *Ultralytics Model Comparison*

URL: <https://docs.ultralytics.com/models/yolo11/#performance-metrics>

Benchmark data showing YOLO11s achieves 18.8ms inference on RTX 4060 at 640px input with 0.38 detections/frame.

1.2 ByteTrack Multi-Object Tracking

[4] ByteTrack: Multi-Object Tracking by Associating Every Detection Box

Source: *ECCV 2022 Paper*

URL: <https://arxiv.org/abs/2110.06864>

Academic paper introducing ByteTrack. Uses Kalman filters and IoU-based Hungarian association. Core tracking algorithm used in IBVAP.

[5] ByteTrack GitHub Repository

Source: *GitHub*

URL: <https://github.com/ifzhang/ByteTrack>

Official implementation with YOLO integration. Provides the tracking pipeline used in IBVAP's per-camera tracker instances.

[6] Kalman Filter for Object Tracking

Source: *SciPy Documentation*

URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve_discrete_are.html

Mathematical foundation for the 8-state Kalman filter used in ByteTrack for motion prediction.

2. Face Recognition Research

2.1 SCRFD Face Detection

[7] SCRFD: Efficient and Accurate Face Detection

Source: InsightFace / arXiv

URL: <https://arxiv.org/abs/2105.03516>

Academic paper on SCRFD (Selective Convolutional Response Face Detection). Used for face localization in IBVAP with configurable input size (320x320 default).

[8] InsightFace GitHub Repository

Source: GitHub

URL: <https://github.com/deepinsight/insightface>

Official implementation of SCRFD and ArcFace. Provides ONNX models used by IBVAP's face detection and recognition pipeline.

[9] ONNX Runtime Documentation

Source: Microsoft ONNX Runtime

URL: <https://onnxruntime.ai/>

Cross-platform inference engine used to run SCRFD and ArcFace models. CPU build used by default; GPU build optional for acceleration.

2.2 ArcFace Face Recognition

[10] ArcFace: Additive Angular Margin Loss for Deep Face Recognition

Source: CVPR 2019 Paper

URL: <https://arxiv.org/abs/1801.07698>

Seminal paper on ArcFace loss for face recognition. Generates 512-dimensional embeddings used in IBVAP for watchlist matching via cosine similarity.

[11] Cosine Similarity for Face Matching

Source: Scikit-learn Documentation

URL: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html

Mathematical method used to compare face embeddings against watchlist entries. Threshold configurable via `FACE_RECOGNITION_THRESHOLD`.

3. ANPR Research

3.1 EasyOCR

[12] EasyOCR: Ready-to-Use OCR with 80+ Languages

Source: [GitHub](#)

URL: <https://github.com/JaidedAI/EasyOCR>

Open-source OCR engine used for ANPR text recognition. Supports English character recognition with confidence scores.

[13] EasyOCR Documentation

Source: [JaidedAI Official Docs](#)

URL: <https://www.jaided.ai/easyocr/documentation/>

Configuration reference for readtext() API, allowlist parameters, and detail output format used in IBVAP's ANPR pipeline.

3.2 Indian Number Plate Standards

[14] Indian Vehicle Registration Plate Format

Source: [Ministry of Road Transport & Highways](#)

URL: <https://morth.nic.in/>

Official format: LL DD LLL DDDD (State Code + District + Letters + Numbers). Used as the grammar for plate validation and OCR error correction.

[15] List of Indian State Registration Codes

Source: [Wikipedia](#)

URL: https://en.wikipedia.org/wiki/Vehicle_registration_plates_of_India

Complete list of 39 valid state/UT codes used in IBVAP's STATE_CODES set for plate validation.

3.3 Temporal Consensus Methods

[16] Temporal Ensemble Methods for Robust Recognition

Source: [IEEE Xplore](#)

URL: <https://ieeexplore.ieee.org/document/8953856>

Research on multi-frame voting for robust recognition. Basis for IBVAP's per-character majority vote consensus algorithm.

[17] D-Fire Dataset for Fire/Smoke Detection

Source: [GitHub](#)

URL: <https://github.com/gaiasd/DFireDataset>

Dataset used to fine-tune YOLOv8n for fire and smoke detection. Contains labeled images of fire and smoke scenarios.

4. Fire & Smoke Detection Research

[18] D-Fire: A Dataset for Fire and Smoke Detection

Source: *GitHub / Research Paper*

URL: <https://github.com/gaiasd/DFireDataset>

Curated dataset for training fire and smoke detection models. Used to fine-tune YOLOv8n with 2 classes (fire, smoke).

[19] YOLOv8 Fine-tuning Guide

Source: *Ultralytics Documentation*

URL: <https://docs.ultralytics.com/modes/train/>

Training methodology used to fine-tune YOLOv8n on the D-Fire dataset for fire/smoke detection in IBVAP.

[20] Fire Detection Using Deep Learning: A Review

Source: *arXiv*

URL: <https://arxiv.org/abs/2301.01805>

Comprehensive review of deep learning approaches for fire detection, informing the choice of YOLOv8n architecture.

5. Tamper-Evident Logging & Cryptographic Integrity

5.1 Hash Chain Data Structures

[21] Hash Chain - Wikipedia

Source: Wikipedia

URL: https://en.wikipedia.org/wiki/Hash_chain

Foundational concept for IBVAP's tamper-evident alert log. Each row stores the SHA-256 hash of the previous row.

[22] SHA-256 Algorithm Specification

Source: NIST FIPS 180-4

URL: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>

NIST standard for the SHA-256 hash function used in IBVAP's chain sealing and evidence integrity.

5.2 Merkle Trees for Verification

[23] Merkle Tree - Wikipedia

Source: Wikipedia

URL: https://en.wikipedia.org/wiki/Merkle_tree

Data structure used for IBVAP's periodic checkpoints. Allows verification of a range of alerts without the full database.

[24] Merkle Tree in Certificate Transparency

Source: RFC 6962

URL: <https://www.rfc-editor.org/rfc/rfc6962>

IETF standard for Merkle tree-based verification, adapted for IBVAP's checkpoint sealing mechanism.

5.3 SQLite WAL Mode

[25] SQLite Write-Ahead Logging

Source: SQLite Official Documentation

URL: <https://sqlite.org/wal.html>

WAL mode provides concurrent reads with single-writer access, used by IBVAP for high-throughput alert logging.

[26] SQLite Database File Format

Source: SQLite Official Documentation

URL: <https://sqlite.org/fileformat2.html>

Reference for the database format used by IBVAP's persistent storage layer.

6. Web Framework & API Research

6.1 FastAPI

[27] FastAPI: Modern, Fast Web Framework for Building APIs

Source: Official Documentation

URL: <https://fastapi.tiangolo.com/>

High-performance async web framework used for all 388 REST endpoints and WebSocket connections in IBVAP.

[28] FastAPI Best Practices

Source: FastAPI Official Documentation

URL: <https://fastapi.tiangolo.com/advanced/best-practices/>

Patterns for async handlers, dependency injection, and background tasks used throughout IBVAP's API layer.

6.2 WebSocket Protocol

[29] RFC 6455: The WebSocket Protocol

Source: IETF Standard

URL: <https://www.rfc-editor.org/rfc/rfc6455>

IETF standard for WebSocket communication used by IBVAP's real-time alert push system.

[30] WebSocket vs REST Polling

Source: MDN Web Docs

URL: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

Reference for choosing WebSocket over polling for real-time alert delivery.

6.3 MJPEG Streaming

[31] MJPEG (Motion JPEG) Streaming

Source: Wikipedia

URL: https://en.wikipedia.org/wiki/Motion_JPEG

Streaming format used by IBVAP's `/stream/{camera_id}` endpoint for real-time video delivery to the dashboard.

7. Computer Vision & Image Processing Research

7.1 OpenCV

[32] OpenCV: Open Source Computer Vision Library

Source: Official Documentation

URL: <https://docs.opencv.org/>

Core library for video capture, frame processing, JPEG encoding, CLAHE enhancement, and overlay rendering in IBVAP.

[33] CLAHE (Contrast Limited Adaptive Histogram Equalization)

Source: OpenCV Documentation

URL: https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

Enhancement technique used for low-light frame processing in IBVAP's night mode pipeline.

7.2 Shapely Geometry

[34] Shapely: Manipulation and Analysis of Geometric Objects

Source: Official Documentation

URL: <https://shapely.readthedocs.io/>

Library used for polygon membership tests, line intersection, and point-in-polygon operations in IBVAP's spatial rules engine.

[35] Point-in-Polygon Algorithms

Source: Wikipedia

URL: https://en.wikipedia.org/wiki/Point_in_polygon

Geometric algorithms underlying IBVAP's zone intrusion detection.

7.3 FFmpeg

[36] FFmpeg: Complete Cross-Platform Solution to Record, Convert and Stream Audio and Video

Source: Official Documentation

URL: <https://ffmpeg.org/documentation.html>

Backend for OpenCV's video capture. Handles RTSP/HTTP stream decoding with configurable timeouts used in IBVAP.

8. Deployment & DevOps Research

8.1 Docker

[37] Docker: Containerization Platform

Source: Official Documentation

URL: <https://docs.docker.com/>

Containerization used for IBVAP's deployment, ensuring consistent operation across different hardware and OS configurations.

[38] Docker Compose

Source: Docker Official Documentation

URL: <https://docs.docker.com/compose/>

Multi-container orchestration used for IBVAP's deployment with volume mounts for evidence storage.

8.2 Edge Computing

[39] Edge Computing: Vision and Challenges

Source: IEEE Xplore

URL: <https://ieeexplore.ieee.org/document/8758303>

Research on edge computing for video analytics, informing IBVAP's design for local processing without cloud dependency.

[40] AI on Edge Devices

Source: NVIDIA Developer Blog

URL: <https://developer.nvidia.com/blog/category/edge-computing/>

Best practices for running AI models on edge hardware, applied to IBVAP's CPU-first architecture with optional GPU acceleration.

9. Night Detection Research

[41] Visual Night Detection vs Clock-Based Methods

Source: Research Gate

URL: <https://www.researchgate.net/publication/334729090>

Research supporting visual darkness detection over clock-based methods, basis for IBVAP's scene-based night detection.

[42] Infrared Night Vision Detection

Source: Wikipedia

URL: https://en.wikipedia.org/wiki/Night_vision

Reference for IR camera detection using saturation and channel spread analysis in IBVAP's night detection pipeline.

[43] Dark Pixel Fraction for Scene Analysis

Source: Google Scholar

URL: <https://scholar.google.com/scholar?q=dark+pixel+fraction+scene+analysis>

Academic research on using dark pixel fraction as a scene brightness indicator, used in IBVAP's darkness scoring.

10. Standards & Compliance

[44] ISO 9241-210: Human-Centred Design for Interactive Systems

Source: ISO Standard

URL: <https://www.iso.org/standard/52075.html>

Human factors standard referenced in IBVAP's UI/UX design for operator dashboards.

[45] MIL-STD-1472H: Human Engineering

Source: Department of Defense

URL: <https://www.dla.mil/>

Military standard for human factors in system design, referenced in IBVAP's operator interface design.

[46] WCAG 2.2 AA: Web Content Accessibility Guidelines

Source: W3C

URL: <https://www.w3.org/TR/WCAG22/>

Accessibility standard referenced in IBVAP's dashboard design for color contrast and non-color indicators.

Summary

IBVAP builds upon 46+ research references spanning object detection (YOLO11), multi-object tracking (ByteTrack), face recognition (SCRFD + ArcFace), optical character recognition (EasyOCR), fire/smoke detection (YOLOv8n D-Fire), cryptographic integrity (SHA-256 hash chains, Merkle trees), web technologies (FastAPI, WebSocket, MJPEG), computer vision (OpenCV, Shapely), and edge deployment (Docker, air-gapped operation). All references are publicly available and were used to inform the architecture, algorithms, and design decisions of the platform.